

UK Patent Application - Description

TECHNICAL FIELD

This invention relates to computer-implemented systems for modelling, predicting, and optimising interpersonal communication outcomes. More specifically, it concerns methods and apparatus for quantifying communication variables across multiple ontological dimensions, generating probabilistic predictions of interaction outcomes through Monte Carlo simulation incorporating user-specific confounding factors and strategic adjustments, recording empirical post-interaction observations, calculating prediction accuracy metrics, and refining future predictions through closed-loop feedback mechanisms.

BACKGROUND ART

Effective interpersonal communication remains critical across professional domains including business development, negotiation, leadership, customer relations, and stakeholder management. Communication outcomes exhibit substantial variability and unpredictability due to complex interactions between cognitive capabilities, emotional states, sociocultural factors, behavioural patterns, and strategic objectives affecting both participants.

Existing technological approaches to communication improvement operate primarily in retrospective analysis mode. Conversation intelligence platforms such as Gong.io and Chorus.ai analyse recorded interactions to identify successful patterns and provide coaching recommendations, but function exclusively post-hoc without predictive capability before interactions occur. Customer relationship management systems including Salesforce and LinkedIn Sales Navigator track historical engagement metrics and relationship timelines, but lack sophisticated models of individual communication characteristics and cannot forecast specific interaction outcomes with quantified probability distributions before meetings take place.

Personality assessment tools including DiSC, Myers-Briggs Type Indicator, and platforms such as Crystal Knows provide static trait profiles based on questionnaire responses or limited behavioural observations. These systems generate broad characterisations but do not quantify situation-specific communication variables, model uncertainty through probability distributions, generate outcome predictions with confidence intervals, or adapt based on observed empirical results from actual interactions.

The fundamental limitations of prior art systems include the absence of comprehensive structured representation of communication variables spanning cognitive, emotional, sociocultural, behavioural, and strategic dimensions; probabilistic simulation engines generating outcome predictions before interactions occur; systematic recording and comparison of predicted versus actual outcomes; calculation of prediction accuracy metrics enabling quantifiable model improvement; and integration of cognitive awareness tools with communication planning workflows.

These limitations result in substantial inefficiency as communicators cannot systematically prepare for interactions with quantified outcome probabilities, test alternative strategies through simulation, identify optimal variable configurations for specific objectives, or benefit from cumulative learning across repeated interactions with empirical validation. There exists a need for a computer-implemented system that models communication variables with mathematical precision, generates probabilistic predictions through simulation accounting for real-time confounding factors, validates predictions against empirical outcomes, calculates measurable accuracy improvements, and provides integrated cognitive reflection capabilities linking psychological patterns to communication effectiveness.

SUMMARY OF INVENTION

The present invention addresses the aforementioned limitations through a computer-implemented system comprising a five-layer variable ontology for comprehensive communication representation; an actor profile data structure encoding individual characteristics; a simulation engine executing Monte Carlo iterations incorporating user-configurable confounding factors and strategic adjustments to predict interaction outcomes; a post-interaction recording module capturing empirical observations; an accuracy calculation engine comparing predictions to actuals; a cognitive distortion logging system integrated with communication planning; and a communication-specific project management framework.

The five-layer ontology quantifies cognitive variables (analytical reasoning, abstract thinking, pattern recognition, knowledge depth, learning rate, cognitive flexibility, memory capacity, processing speed), emotional variables (emotional intelligence, empathy, stress resilience, emotional stability, optimism bias, expressiveness, conflict tolerance), sociocultural variables (cultural fluency, social status sensitivity, authority response patterns, group identity strength, formality preference, network centrality), behavioural variables (communication style, verbosity, interruption tendency, question frequency, risk tolerance, decision speed, detail orientation), and strategic variables (goal clarity, strategic thinking, long-term orientation, negotiation skill, adaptability). Actor profiles store estimated values for these variables based on observable cues, with confidence scores tracking estimation uncertainty.

The system enables users (termed "Architects") to configure three core skills (Observation for accurately reading actor variables, Adaptation for flexibly adjusting strategies, Self-Awareness for recognising own biases and limitations) with values typically ranging from zero point three to zero point nine on a normalised zero-to-one scale. Five confounding variables modify effective skill levels in real-time: Stress Level reduces observation effectiveness by thirty percent and adaptation effectiveness by thirty percent; Fatigue Level reduces observation by twenty percent and adaptation by twenty percent; Emotional State reduces observation by twenty percent and self-awareness by thirty percent; Overconfidence reduces self-awareness by forty percent; Ego Investment reduces adaptation by thirty percent. These penalties compound multiplicatively such that an Architect with base observation skill of zero point eight experiencing stress level zero point six and fatigue level zero point four would have effective observation of approximately zero point five.

The simulation engine executes Monte Carlo iterations, typically one thousand runs but configurable from one hundred to ten thousand, sampling from actor variable distributions and applying confounder-adjusted skills to generate outcome predictions across three strategic dimensions: change in Notoriety, change in Respect, and change in Wealth. Five strategy levers (Warmth, Competence, Dominance, Status, Rapport) enable architects to configure intended communication approach on bipolar scales from negative one to positive one, with these adjustments incorporated into outcome calculation functions.

Novel technical contributions include a cognitive distortion logging system integrated with communication planning that records observed psychological elements, links episodes to specific actors who triggered them, creates Jungian polarity integration experiments as communication tasks, and feeds self-awareness metrics into skill calculations. A communication-specific project management architecture organises strategic interaction tasks rather than generic deliverables, with pre-configured templates for stakeholder alignment, conflict resolution, and relationship building scenarios. Post-interaction recording captures both predicted and actual values for confounders, strategy levers, and seven strategic effectiveness indexes, calculates prediction accuracy metrics, and identifies systematically misestimated variables for focused improvement.

DETAILED DESCRIPTION

The invention is embodied in a computer system comprising processing hardware including central processing unit with floating-point arithmetic capabilities for Monte Carlo simulation, memory storage including random access memory for simulation execution and persistent database storage for actor profiles and historical interaction data, network communication interfaces providing application programming interface endpoints for data input and output, and user interface components comprising web-based visualisation dashboard accessible through standard internet browsers.

Figure 1 illustrates the four-stage predictive learning cycle forming the core feedback loop of the invention. The PREDICT stage employs Monte Carlo simulation with Beta distributions to generate probabilistic outcome forecasts incorporating uncertainty quantification (innovation ①, Claims 1, 7, 14). The INTERACT stage executes real-world communication events according to configured strategies. The OBSERVE stage systematically records actual confounders, strategies, and outcomes from completed interactions. The COMPARE stage calculates prediction accuracy by comparing forecasted versus actual outcomes and identifies systematic biases in the prediction model, enabling closed-loop learning (innovation ②, Claims 1, 11, 24, 25). This empirical validation cycle enables continuous model improvement over time, with each interaction providing new training data for refining subsequent predictions. The cycle operates continuously throughout system usage, creating a self-improving prediction engine that becomes more accurate as the user completes more interactions.

The backend implementation utilises FastAPI framework providing representational state transfer application programming interface endpoints, PostgreSQL relational database version thirteen or higher for persistent storage with JSONB columns enabling flexible semi-structured data storage, NumPy and SciPy libraries for numerical computation, and SQLAlchemy object-relational mapping for database interactions. The frontend comprises

a React single-page application implemented in TypeScript with real-time WebSocket connections for simulation progress updates and data visualisation using D3.js library for interactive charts and Recharts library for statistical graphics.

The system optimises three user-configurable strategic outcomes measured across the architect's entire network of actor relationships. These outcomes provide quantitative targets for communication strategy rather than relying on subjective assessments of interaction quality.

Notoriety, denoted N , quantifies the degree to which relevant actors know, understand, and would support the architect's aims. The calculation aggregates three components for each actor in the network. Awareness measures whether the actor knows who the architect is and what they do, ranging from zero (no knowledge) through zero point five (knows name and role but limited context) to one (comprehensive knowledge of architect's work). Understanding measures whether the actor accurately comprehends the architect's strategic aims, ranging from zero (misconceptions or no understanding) through zero point five (partial understanding with gaps) to one (accurate comprehensive understanding). Support willingness measures whether the actor would actively help if asked, ranging from zero (would refuse or actively oppose) through zero point five (neutral or conditionally supportive) to one (would enthusiastically support).

The notoriety formula aggregates these components across all relevant actors:

$$N = (1/n) \times \text{Sum}(i=1 \text{ to } n) [\text{awareness}(i) \times \text{understanding}(i) \times \text{support_willingness}(i)]$$

where n represents the total number of relevant actors in the network. A notoriety value approaching zero indicates the architect is unknown or misunderstood in relevant networks, a value of approximately zero point five indicates moderate recognition with some support potential, and a value approaching one indicates widespread recognition, accurate understanding, and active support from key actors.

Change in notoriety from a single interaction, denoted ΔN , represents the incremental improvement or degradation resulting from that specific communication event. Interactions that successfully introduce the architect to new actors, correct misconceptions, or demonstrate value increase notoriety. Interactions that create negative impressions or fail to clarify aims decrease notoriety.

Respect, denoted R , quantifies the degree to which actors feel understood, valued, and believe their wants, needs, and desires are considered by the architect. This outcome measures relationship quality and mutual regard rather than mere awareness. The calculation aggregates three components analogous to notoriety. Feeling understood measures whether the actor feels the architect comprehends their perspective, ranging from zero (feels misunderstood, dismissed, or invisible) to one (feels deeply understood and validated). Consideration measures whether the actor believes their priorities are acknowledged, ranging from zero (priorities ignored or dismissed) to one (full recognition and consideration of actor's goals). Value alignment measures whether the actor perceives mutual benefit in engagement, ranging from zero (sees zero-sum relationship or exploitation) to one (sees strong win-win potential).

$$R = (1/n) \times \text{Sum}(i=1 \text{ to } n) [\text{feels_understood}(i) \times \text{consideration}(i) \times \text{value_alignment}(i)]$$

Change in respect, Δ_R , represents incremental improvement or degradation from specific interactions. Interactions demonstrating empathy, acknowledging actor concerns, and finding mutual benefits increase respect. Interactions appearing dismissive, self-serving, or tone-deaf decrease respect.

Financial Wealth, denoted W , quantifies economic value generation through salary increases, asset growth, cashflow improvements, opportunity pipeline development, and executed project value realisation. Unlike notoriety and respect which normalise to zero-one range, wealth is measured in absolute currency units, typically pounds sterling, and can span wide ranges from thousands to millions depending on the architect's career stage and objectives.

Direct wealth, W_{direct} , includes salary increases (annual compensation growth in pounds or percentage change), asset growth (appreciating investments, property, equity stakes), and cashflow increases (net monthly or annual cash improvement). Pipeline wealth, W_{pipeline} , represents expected value of opportunities currently in progress, calculated as the sum across all opportunities of opportunity value multiplied by probability of execution. For example, an opportunity valued at fifty thousand pounds with seventy percent execution probability contributes thirty-five thousand pounds to pipeline wealth. Executed wealth, W_{executed} , represents realised gains from completed projects such as revenue increases, cost reductions, or investment returns.

The system enables architects to assign priority weights to each outcome based on current strategic focus, with weights denoted w_N , w_R , and w_W subject to the constraint that $w_N + w_R + w_W$ equals one. An architect pursuing career advancement might configure weights of w_N equals zero point three, w_R equals zero point two, w_W equals zero point five, emphasising financial outcomes while maintaining moderate attention to visibility and relationships. An architect focused on relationship building might instead use w_N equals zero point two, w_R equals zero point six, w_W equals zero point two. These weights directly influence the ROI Index calculation described subsequently.

Figure 6 illustrates the seven strategic effectiveness indexes providing multi-dimensional interaction assessment. The system measures interaction effectiveness across seven strategic dimensions (ROI Index, Confidence Delta, Resistance Index, Energy Cost, Influence Depth, Leverage Activation, and Narrative Coherence), providing multi-faceted assessment beyond simple success/failure binary outcomes (innovation ①, Claims 1, 9, 20). These indexes enable architects to diagnose interaction quality, identify improvement areas, and optimise process as well as results.

The ROI Index quantifies overall strategic value from an interaction, calculated as weighted sum of outcome changes scaled to zero-to-one-hundred range for interpretability. The formula is:

$$\text{ROI_index} = 50 + (50 \times [w_N \times \Delta_N + w_R \times \Delta_R + w_W \times \Delta_W_{\text{normalised}}])$$

where Δ_N , Δ_R represent changes in notoriety and respect (already normalised to approximately negative one to positive one range), $\Delta_W_{\text{normalised}}$ represents change in wealth normalised by dividing by maximum possible wealth change for that interaction, and weights w_N , w_R , w_W reflect current strategic priorities. The baseline value of fifty represents neutral outcome with no change. Values above seventy-five

indicate highly productive interactions with strong outcome advancement, values between fifty and seventy-five indicate moderately positive progress, values between forty and fifty indicate slightly positive outcomes with minimal gains, values between twenty-five and forty indicate slightly negative outcomes with setbacks, and values below twenty-five indicate highly negative interactions causing significant damage to strategic outcomes. The system tracks ROI Index across all interactions to identify high-value communication patterns and low-value activities requiring strategy adjustment.

Confidence Delta, denoted Δ_C , quantifies change in the architect's confidence about actor variable estimates resulting from the interaction. Before interaction, the architect holds beliefs about actor variables with associated uncertainty. During interaction, observations may confirm existing estimates (high Δ_C positive), reveal unexpected characteristics (high Δ_C negative indicating previous misconceptions), or provide ambiguous signals (Δ_C near zero). The calculation compares estimated variable values before interaction to updated estimates after incorporating observed behaviours. High positive confidence delta indicates the interaction successfully reduced uncertainty through clear behavioural signals. Negative confidence delta indicates the interaction introduced confusion or revealed previous estimates were inaccurate. This metric enables the system to track the architect's improving observation skills over time, as more experienced architects achieve consistently positive confidence delta through accurate initial estimates subsequently confirmed.

Resistance Index, denoted $I_{\text{resistance}}$, quantifies degree of friction, pushback, or misalignment encountered during interaction on zero-to-one scale. The calculation aggregates observed resistance signals including verbal objections (phrases such as "I don't think that will work" or "We tried that before"), non-verbal cues (crossed arms, leaning back, frowning), avoidance behaviours (deflection, topic changing, meeting postponement), delayed responses (slow replies, missed deadlines), and passive aggression (compliance without commitment). Each resistance signal is assigned an intensity value from zero point two (mild skepticism easily addressed) through zero point five (moderate pushback requiring negotiation) to one point zero (dealbreaker with complete refusal). The resistance index sums frequency multiplied by intensity for all observed signals, normalised to zero-one range. Values approaching zero indicate smooth aligned interaction with high receptivity, values around zero point five indicate moderate friction requiring skilled navigation, and values approaching one indicate high opposition or fundamental misalignment. The system uses resistance index to recommend alternative strategies for high-resistance actors and identify scenarios requiring additional preparation or different approaches.

Energy Cost, denoted E_{cost} , quantifies cognitive and emotional load required for the interaction on zero-to-one scale. The calculation combines cognitive load (mental effort from information density, decision difficulty, novel concepts, and concurrent concerns) and emotional load (effort to manage own emotional state, navigate interpersonal tension, and maintain composure under pressure). The formula is:

where cognitive_load incorporates complexity (information density and decision difficulty), unfamiliarity (inverse of actor familiarity requiring more mental effort), and multitasking (number of concurrent objectives or concerns), and emotional_load incorporates stress (perceived stakes and anxiety), regulation effort (energy expended managing emotional reactions), and tension (interpersonal friction or discomfort). Additionally, confounding

factors modify energy cost such that low energy levels, high fatigue, high stress, and physical exhaustion increase perceived energy cost for equivalent interactions. Values approaching zero indicate effortless energising interactions that leave the architect feeling refreshed, values around zero point five indicate moderate effort consistent with professional norms, and values approaching one indicate exhausting draining interactions causing significant fatigue. The system tracks energy cost relative to ROI Index to identify inefficient interactions (high energy cost, low ROI) requiring strategy adjustment and efficient interactions (low energy cost, high ROI) worth replicating.

Influence Depth, denoted I_{depth} , quantifies degree of internal belief or behaviour change achieved in the actor on zero-to-one scale. Superficial influence might change an actor's stated position without changing underlying beliefs (low influence depth), whereas deep influence shifts fundamental perspectives or behavioural patterns (high influence depth). The system distinguishes compliance (actor agrees but without conviction), identification (actor accepts architect's position because they respect the source), and internalisation (actor genuinely adopts new beliefs as their own). Values approaching zero indicate no lasting impact beyond polite acknowledgment, values around zero point five indicate moderate influence such as stated agreement or tentative behaviour change, and values approaching one indicate profound shifts in actor's worldview or sustained behaviour modification. This metric enables architects to assess whether interactions achieve surface-level compliance or genuine transformation.

Narrative Coherence, denoted $I_{\text{narrative}}$, quantifies consistency and clarity of the architect's message across the interaction on zero-to-one scale. High narrative coherence indicates the architect maintained a clear compelling story connecting all discussion points to overarching themes. Low narrative coherence indicates scattered messaging, contradictions, or unclear connections between topics. The calculation assesses message clarity (how easily the actor could summarise the architect's key points), internal consistency (absence of contradictions or confusing shifts), thematic unity (how well individual points connected to central narrative), and memorable structure (whether the interaction followed a coherent beginning-middle-end progression). Values approaching zero indicate confusing disjointed communication leaving the actor uncertain about main messages, values around zero point five indicate functional communication with room for improved clarity, and values approaching one indicate exceptionally clear memorable messaging that the actor could easily explain to others. This metric helps architects identify whether poor interaction outcomes stem from strategic content issues or presentation and delivery problems.

Figure 11 illustrates the multiplicative confounder penalty approach contrasting with conventional additive methods. The system incorporates five confounding variables that degrade architect skill effectiveness during interactions, providing realistic simulation accounting for human performance limitations under sub-optimal conditions. These confounders operate through multiplicative penalty factors (innovation ①, Claims 1, 8, 15, 19) applied to base skill levels, ensuring results remain bounded between zero and one and accurately modeling compound degradation effects, unlike additive approaches that can produce unrealistic unbounded values.

Stress Level, ranging from 0 (completely relaxed) to 1 (maximum stress), reduces observation skill effectiveness by 30% and adaptation skill effectiveness by 30%. The

effective observation skill becomes:

$$\text{effective_observation} = \text{base_observation} \times (1 - \text{stress} \times 0.3)$$

$$\text{effective_adaptation} = \text{base_adaptation} \times (1 - \text{stress} \times 0.3)$$

An architect with base observation skill of 0.8 experiencing stress level 0.6 would have effective observation = $0.8 \times (1 - 0.18) \approx 0.66$. High stress impairs ability to detect subtle behavioural cues and reduces cognitive flexibility for strategy adjustment.

Fatigue Level, ranging from 0 (fully rested) to 1 (exhausted), reduces observation skill by 20% and adaptation skill by 20% through analogous multiplicative penalties. An architect operating with fatigue level 0.5 experiences:

$$\text{effective_observation} = \text{base_observation} \times (1 - 0.1)$$

retaining 90% of base capability. Cumulative fatigue from back-to-back meetings compounds these effects.

Emotional State, ranging from 0 (highly negative emotional state) to 1 (highly positive emotional state), reduces observation skill by 20% and self-awareness skill by 30% when in negative emotional states. The calculation uses $(1 - \text{emotional_state})$ to convert the scale such that low emotional state values create high penalties. An architect in negative emotional state ($\text{emotional_state} = 0.3$) experiences effective observation reduced by 14% and self-awareness reduced by 21%. Negative emotional states cause inward focus reducing capacity to accurately read others and tendency toward defensiveness reducing self-awareness.

Overconfidence, ranging from 0 (appropriately confident) to 1 (extremely overconfident), reduces self-awareness skill by 40%. The penalty calculation follows the pattern:

$$\text{effective_self_awareness} = \text{base_self_awareness} \times (1 - \text{overconfidence} \times 0.4)$$

An architect with overconfidence level 0.7 retains only 72% of base self-awareness. Overconfidence creates blind spots preventing recognition of own limitations and biases.

Ego Investment, ranging from 0 (emotionally detached from outcome) to 1 (heavily ego-invested), reduces adaptation skill by 30%. High ego investment in being right or winning creates rigidity preventing flexible strategy adjustment. An architect with ego investment level 0.8 retains only 76% of base adaptation capability.

These confounders combine multiplicatively rather than additively, reflecting compounding effects of multiple stressors. An architect with base observation skill of 0.7, experiencing stress level 0.4, fatigue level 0.5, and negative emotional state ($\text{emotional_state} = 0.4$), would have effective observation calculated as:

$$\text{effective_observation} = 0.7 \times (1 - 0.4 \times 0.3) \times (1 - 0.5 \times 0.2) \times (1 - (1 - 0.4) \times 0.2)$$

Thus multiple moderate confounders reduce the architect from 70% base capability to 49% effective capability, demonstrating substantial performance degradation requiring either confounder mitigation (stress reduction, rest, emotional regulation) or acknowledgment of temporarily reduced capacity.

Figure 10 illustrates the three-level project organization hierarchy enabling structured management of complex multi-stage communication initiatives. The PROJECT level defines strategic objectives and links relevant actors (Claims 4, 5, 13). The TASK level organizes phased workflows breaking complex objectives into manageable steps. The INTERACTION level provides pre-configured templates for specific interaction types. The system supports specialized project types including stakeholder alignment, conflict resolution, relationship building, and negotiation scenarios (innovation ①, Claim 5), each with tailored workflow structures optimized for that communication objective.

Figure 12 illustrates the five strategy levers with data-driven recommended settings and real-time Monte Carlo preview capabilities. The system provides five bipolar strategy levers (warmth, competence, dominance, status, rapport) that the user configures prior to each interaction to shape their communication approach. Each lever is represented on a continuous scale from negative one to positive one, allowing nuanced positioning between opposing extremes. The system provides recommended lever settings based on actor characteristics and scenario context (innovation ①, Claims 1, 11), and displays live probability distribution previews as levers are adjusted (innovation ②, Claim 7).

Warmth Lever (negative one equals cold/formal to positive one equals warm/personal) controls emotional tenor. Negative values emphasize professional distance, strict formality, and emotional restraint. Positive values emphasize personal connection, warmth, and emotional expressiveness. Selection depends on actor characteristics and scenario context.

Competence Lever (negative one equals humble/uncertain to positive one equals confident/expert) controls expertise projection. Negative values signal openness to learning, deference to others' expertise, and acknowledgment of uncertainty. Positive values project confidence, subject matter expertise, and authoritative knowledge. Calibration depends on actual expertise levels, actor skepticism, and power dynamics.

Dominance Lever (negative one equals submissive/yielding to positive one equals dominant/assertive) controls behavioral assertiveness. Negative values involve yielding conversational control, following the other party's lead, and accommodating preferences. Positive values involve directing the conversation, setting the agenda, and asserting preferences firmly.

Status Lever (negative one equals low status signaling to positive one equals high status signaling) controls social positioning independent of actual status. Negative values involve deferential language, seeking approval, and acknowledging the other party's superior position. Positive values involve status assertion through language choices, expectation-setting, and framing that positions the user as higher status.

Rapport Lever (negative one equals minimal rapport building to positive one equals maximum rapport building) controls investment in relationship-building activities distinct from warmth. Negative values minimize small talk, personal disclosure, and commonality exploration. Positive values emphasize discovering shared interests, personal storytelling, and activities that build interpersonal connection.

These five strategy levers are mathematically distinct from actor variables. Variables describe enduring characteristics of individuals, while strategy levers represent tactical choices made by the user for a specific interaction. The Monte Carlo simulation

incorporates strategy lever settings as inputs to outcome functions, enabling the system to model how different strategic approaches affect predicted outcomes given the actor's variable profile.

Figure 9 illustrates the cognitive journal integration system providing unique psychological self-awareness capabilities. A particularly novel component of the invention is the integration of a cognitive distortion logging system with the communication simulation framework. This integration serves three technical functions: tracking cognitive bias episodes that may impair accurate actor assessment and simulation quality, linking distortion episodes to specific actors to identify bias patterns, and feeding self-awareness metrics derived from distortion logging into the architect skill calculation framework.

The cognitive journal allows the user to log episodes where they recognize cognitive distortions occurring during interaction preparation, execution, or reflection. Each logged distortion episode records observed elements including triggering external stimulus, associated somatic feelings, interpreted core belief, and positive reframe through a four-stage logging process (innovation ①).

Actor Linkage: Each cognitive journal entry can be linked to a specific actor via a foreign key relationship stored in database field `linked_actor_id` (Claim 2, innovation ①). This linkage enables the system to identify which actors or actor types systematically trigger cognitive distortions for the user. For example, analysis might reveal that the user experiences overconfidence distortions when interacting with actors assessed as having low analytical reasoning, or experiences catastrophic thinking when interacting with high-status actors.

Jungian Polarity Integration: The system maps logged distortions to Jungian psychological polarities such as autonomy versus connection, assertion versus receptivity, and perfection versus acceptance. When a distortion is logged, the system can generate a suggested integration experiment as a task (innovation ②, Claim 3), encouraging the user to explore the opposite pole. For instance, if distortion reflects extreme autonomy orientation, generate a connection-focused exercise. This provides a pathway for psychological development that directly supports improved communication effectiveness.

Self-Awareness Metric Feeding: The frequency, recognition speed, and successful reframing of cognitive distortions are aggregated into a self-awareness score. This score is incorporated into the architect skill framework, specifically affecting the self-awareness dimension. Higher self-awareness (demonstrated through consistent cognitive distortion recognition and reframing) improves simulation accuracy by reducing bias in actor variable estimation and strategy selection.

The cognitive journal integration represents a significant innovation beyond generic cognitive behavioral therapy logging tools. By explicitly linking psychological bias recognition to actor profiles and feeding self-awareness metrics into communication simulation parameters, the system creates a closed feedback loop between psychological development and strategic effectiveness.

The system implements a specialized task management framework called Communication Projects, distinct from generic project management tools, designed specifically for planning and tracking strategic communication initiatives. A Communication Project

represents a coherent communication objective requiring multiple interactions, persistent actor relationship management, and strategic coherence across time.

Project Structure: Each Communication Project contains a strategic objective such as securing series A funding from three target venture capitalists, resolving organizational conflict with engineering leadership, or building strategic partnership with specific company. Projects include linked actors (the set of actors involved in achieving the objective), task hierarchy (individual interactions and preparation activities structured as tasks explicitly typed as communication events), outcome templates (pre-configured simulation templates for common communication scenarios), and progress tracking (aggregated metrics showing project-level strategic outcome trajectories across all linked actors).

Project Types: The system includes specialized project types with domain-specific templates. Stakeholder Alignment Projects manage communication with multiple stakeholders requiring buy-in for an initiative such as product launch or organizational change. Conflict Resolution Projects provide structured approach to resolving interpersonal or group conflicts through phased communication. Relationship Building Projects systematically develop strategic relationships such as mentor cultivation or strategic partnership development. Negotiation Projects handle complex multi-stage negotiations requiring preparation for multiple interaction rounds.

The Communication Projects architecture provides strategic coherence across interactions. Because projects track actor relationships longitudinally, the system can identify when relationship degradation in one interaction threatens project objectives, trigger alerts for relationship recovery actions, and recommend actor-specific strategies that maintain project-level strategic alignment. This represents an inventive integration of task management with probabilistic communication simulation not present in prior art.

Figure 2 illustrates the comprehensive five-layer actor variable ontology providing the multi-dimensional actor modeling framework that feeds the outcome prediction functions. The system employs this ontology categorising human attributes and states into five hierarchical layers totaling thirty-four distinct variables (innovation ①, Claim 6). This structure enables the calculation of Delta_Notoriety, Delta_Respect, and Delta_Wealth through multi-dimensional actor modeling rather than simple trait-based prediction (innovation ②, Claims 1, 21). The ontology is implemented as a PostgreSQL JSONB structure enabling flexible storage and querying.

Cognitive Variables (eight variables): analytical reasoning, abstract thinking, pattern recognition, knowledge depth, learning rate, cognitive flexibility, memory capacity, processing speed

Emotional Variables (seven variables): emotional intelligence, empathy, stress resilience, emotional stability, optimism bias, emotional expressiveness, conflict tolerance

Sociocultural Variables (six variables): cultural fluency, social status sensitivity, authority response patterns, group identity strength, formality preference, network centrality

Behavioural Variables (seven variables): communication style, verbosity, interruption tendency, question frequency, risk tolerance, decision speed, detail orientation

Strategic Variables (six variables): goal clarity, strategic thinking, long-term orientation, negotiation skill, adaptability, relationship motivation

Each variable is stored as a data structure containing alpha parameter, beta parameter, observation count tracking number of empirical updates, and last_updated timestamp. For example, an actor's empathy variable might be represented as {"empathy": {"alpha": 15.0, "beta": 5.0, "observations": 8, "last_updated": "2024-03-15"}}.

Figure 3 illustrates the key distinction between the invention's probabilistic variable representation using Beta distributions versus conventional scalar approaches. The probabilistic representation enables uncertainty quantification, confidence intervals, and Bayesian updating (innovation ①, Claims 1, 10, 14, 22), contrasting with prior art systems that store only single point estimates lacking any measure of estimation confidence. This structured representation enables probabilistic reasoning about actor characteristics. When the Monte Carlo simulation samples from actor variable distributions, it draws from Beta(alpha, beta) for each variable, generating a complete actor state vector for that simulation iteration. Repeated sampling across thousands of iterations produces outcome probability distributions accounting for estimation uncertainty.

Figure 4 illustrates the assessment interface providing the bridge between natural language actor assessment and rigorous probabilistic modeling. The system provides a slider-based assessment interface for actor profile creation. For each variable in the ontology, the user provides best estimate using a slider control ranging from zero to one hundred (mapped internally to zero to one scale), and confidence level using a qualitative selector including low confidence (generates high variance Beta distribution), medium confidence (generates moderate variance), and high confidence (generates low variance concentrated distribution). The interface automatically converts these intuitive inputs into Beta distribution parameters (innovation ①, Claims 1, 10) which are stored in PostgreSQL JSONB format (Claim 22).

The backend converts these inputs into Beta distribution parameters using method of moments calculation. Given user-specified mean μ and desired variance σ^2 corresponding to confidence level, the alpha and beta parameters are calculated as:

For example, if user estimates an actor's analytical reasoning as 70% ($\mu = 0.7$) with medium confidence ($\sigma^2 = 0.02$), the system calculates $\alpha \approx 24$ and $\beta \approx 10.2$. This Beta(24, 10.2) distribution has mean 0.7 and moderate spread reflecting medium confidence.

Figure 5 illustrates the four-stage Monte Carlo simulation engine process. The core technical innovation of the simulation engine is its ability to generate probabilistic outcome predictions by executing repeated sampling from actor variable distributions (stage 1, Claims 1, 7, 14) while incorporating real-time architect state confounders through multiplicative penalty factors (stage 2, innovation ①, Claims 1, 8, 15, 19), calculating outcome predictions (stage 3), and aggregating results with confidence intervals over typically one thousand iterations (stage 4).

Input: Architect architect_state (including base skills and current confounders), Actor actor_profile (with Beta distributions for all variables), Strategy strategy_levers (five

dimensions from -1 to +1), Scenario scenario_context, Integer iteration_count (default 1000)

Step 1: Calculate effective architect skills by applying confounder penalties multiplicatively:

$$\text{effective_observation} = \text{base_observation} \times (1 - \text{stress} \times 0.3) \times (1 - \text{fatigue} \times 0.2) \times (1 - (1 - \text{emotional_state}) \times 0.2)$$
$$\text{effective_adaptation} = \text{base_adaptation} \times (1 - \text{stress} \times 0.3) \times (1 - \text{fatigue} \times 0.2) \times (1 - \text{ego_investment} \times 0.3)$$
$$\text{effective_self_awareness} = \text{base_self_awareness} \times (1 - (1 - \text{emotional_state}) \times 0.3) \times (1 - \text{overconfidence} \times 0.4)$$

Step 2: Initialize results array with capacity for iteration_count outcome records.

Step 3: For each iteration i from 1 to iteration_count execute sampling loop. For each variable v in actor_profile.variables, sample actor_variables[v] from Beta distribution with parameters $\alpha = \text{actor_profile.variables}[v].\alpha$ and $\beta = \text{actor_profile.variables}[v].\beta$ using NumPy random.beta function. Calculate seven strategic indexes using effective architect skills, sampled actor variables, strategy lever settings, and scenario context. Compute change in notoriety $\Delta_N = \text{calculate_notoriety_change}$ with inputs effective observation, effective adaptation, sampled actor variables, strategy levers. Compute change in respect $\Delta_R = \text{calculate_respect_change}$ with inputs effective adaptation, effective self_awareness, sampled actor variables, strategy levers. Compute change in wealth $\Delta_W = \text{calculate_wealth_change}$ with inputs effective skills, sampled actor variables, strategy levers, scenario context. Store results in results array index i .

Step 4: Aggregate results across all iterations. Calculate mean values mean_delta_N = mean of results array delta_N column, mean_delta_R = mean of delta_R column, mean_delta_W = mean of delta_W column. Calculate percentile-based confidence intervals using NumPy percentile function with arguments results array delta_N column and percentiles [2.5, 97.5]. Repeat for delta_R and delta_W. Compile strategic indexes aggregating means and variances for ROI Index, Confidence Delta, Resistance Index, Energy Cost, Influence Depth, Leverage Activation, and Narrative Coherence.

Output: Predictions object containing mean outcome changes (mean_delta_N, mean_delta_R, mean_delta_W), ninety-five percent confidence intervals for each outcome, aggregated strategic index predictions with uncertainty ranges, and iteration count used for result generation.

The simulation engine leverages NumPy vectorized operations for computational efficiency. Sampling ten thousand values from a Beta distribution requires approximately 0.002 seconds on modern hardware. Total execution time for one thousand iterations with thirty-four variables per actor scales approximately linearly, completing in 0.4 to 0.5 seconds, thereby supporting interactive real-time use.

After an actual interaction completes, the system captures empirical observations enabling comparison between predicted and actual outcomes. This feedback loop serves two critical functions: quantifying prediction accuracy to track model improvement over time, and identifying systematically misestimated variables requiring recalibration or additional observation.

Figure 7 illustrates the post-interaction recording interface implemented as a modal dialog component. The interface presents dual input sections for actual confounders experienced, actual strategy levers deployed, actual strategic indexes observed, and actual outcomes achieved.

Actual Confounders Section: For each of the five confounders (stress, fatigue, emotional state, overconfidence, ego investment), the user adjusts a slider from zero to one hundred indicating the level actually experienced during the interaction. These values may differ from pre-interaction predictions if unexpected stressors arose or anticipated anxiety did not materialize.

Actual Strategy Levers Section: For each of the five strategy levers (warmth, competence, dominance, status, rapport), the user adjusts a bipolar slider from negative one hundred to positive one hundred indicating the approach actually deployed. These may differ from planned settings if the interaction dynamics required tactical adjustment.

Actual Strategic Indexes Section: The user provides subjective assessments or objective measurements where available for each of the seven strategic indexes. ROI Index can be calculated automatically if outcome changes are provided. Resistance Index relies on user assessment of friction encountered. Energy Cost reflects subjective fatigue experienced. Influence Depth, Leverage Activation, and Narrative Coherence require qualitative judgment about interaction quality.

Actual Outcomes Section: The user records actual change in notoriety (did the actor's awareness, understanding, or support willingness increase as predicted), actual change in respect (did the relationship quality improve as anticipated), and actual change in wealth (did financial outcomes such as deal closure, pipeline advancement, or opportunity creation occur as forecasted). Additionally, interaction duration and free-text notes enable qualitative context capture.

The backend processes this recorded data by calculating accuracy metrics comparing predicted versus actual values across all dimensions. Mean Absolute Error is calculated as:

summed across all outcome dimensions. For continuous outcomes such as Delta_N, Delta_R, Delta_W, this is straightforward arithmetic. For categorical outcomes, the system uses cross-entropy or categorical accuracy metrics. The system maintains cumulative accuracy statistics enabling time-series analysis of prediction improvement. Graph visualizations show accuracy trends revealing whether the architect's predictive capability improves with experience.

Figure 8 illustrates the multi-dimensional accuracy analysis dashboard tracking prediction errors across four key dimensions: outcome prediction errors, index prediction errors, skill estimation errors, and systematic bias patterns. This comprehensive tracking across all system dimensions (innovation ①, Claims 1, 11, 24, 25) enables closed-loop learning and continuous model improvement. Systematic bias identification analyzes which variables consistently produce overestimation or underestimation. If an architect repeatedly overestimates their effective adaptation skill under stress, the system flags this pattern and recommends either recalibration of base adaptation estimate or increased attention to stress confounder severity. If outcomes consistently underperform predictions when interacting with high-status actors, the system identifies status-related blind spots requiring

focused improvement.

Example Scenario: Stakeholder Alignment Communication

To illustrate the complete workflow, consider an architect preparing for a stakeholder alignment interaction. The architect is a product manager seeking approval from an engineering director (the actor) to prioritize a new feature in the development roadmap.

The architect creates an actor profile for the engineering director using the slider-based assessment interface. Based on previous meetings and email exchanges, the architect estimates analytical reasoning at eighty (high logical rigor), empathy at forty (task-focused, limited emotional warmth), formality preference at seventy (prefers structured communication), risk tolerance at thirty (conservative, prefers proven approaches), and strategic thinking at seventy-five (long-term oriented). Confidence levels are set to medium for most variables, generating Beta distributions with moderate variance.

The architect assesses their current state. Base skills are observation 0.7, adaptation 0.6, self-awareness 0.65. Current confounders are stress level 0.5 (moderate anxiety about securing approval), fatigue 0.3 (well-rested), emotional state 0.6 (slight anxiety but generally positive), overconfidence 0.2 (appropriately humble given uncertainty), ego investment 0.4 (some personal attachment to the feature but manageable). Effective skills are calculated applying confounder penalties: effective observation approximately 0.61, effective adaptation approximately 0.51, effective self-awareness approximately 0.6.

The architect configures strategy levers. Warmth set to -0.2 (slightly formal professional tone matching actor's formality preference). Competence set to +0.6 (confident subject matter expertise without arrogance). Dominance set to 0 (collaborative balanced power dynamic). Status set to -0.1 (slight deference acknowledging director's authority). Rapport set to +0.3 (moderate investment in relationship building without excessive personal talk).

The system executes one thousand iterations sampling from the director's variable distributions and calculating predicted outcomes. Results predict mean change in notoriety Δ_N of +0.12 (modest awareness increase), mean change in respect Δ_R of +0.08 (slight relationship improvement), change in wealth Δ_W of estimated five thousand pounds if feature approval leads to customer retention (thirty percent probability). Strategic indexes predict ROI Index of 62 (moderately positive), Resistance Index of 0.45 (moderate pushback expected given risk aversion), Energy Cost of 0.55 (effortful interaction requiring careful navigation), Confidence Delta of +0.1 (expect to confirm existing estimates), Influence Depth of 0.4 (likely to achieve stated agreement without deep belief change), and Narrative Coherence of 0.65 (clear structured message planned).

The architect conducts the meeting following the planned strategy. During the interaction, the director raises expected objections about implementation complexity (resistance signal) but responds positively to data showing customer demand (competence lever effectiveness). The architect maintains composure despite initial pushback (stress confounder managed). Conversation concludes with conditional approval pending technical feasibility assessment.

Post-interaction, the architect records actual values. Actual confounders: stress 0.6 (slightly higher than anticipated due to unexpected objection intensity), fatigue 0.4, emotional state 0.55, overconfidence 0.2, ego investment 0.5 (increased during defensive moments). Actual strategy levers: warmth -0.1 (slightly warmer than planned when

building rapport), competence +0.7 (stronger expertise projection when presenting data), dominance +0.2 (more assertive than planned when addressing objections), status 0, rapport +0.4. Actual outcomes: Delta_N +0.15 (director now understands feature value better than expected), Delta_R +0.05 (slight relationship improvement, less than predicted due to tension moments), Delta_W five thousand pounds pipeline (conditional approval achieved). Strategic indexes: ROI Index 68 (better than predicted due to higher notoriety gain), Resistance Index 0.5 (slightly higher friction than expected), Energy Cost 0.6 (more draining than anticipated), Confidence Delta +0.13 (confirmed estimates, slightly higher learning), Influence Depth 0.35 (less deep influence than hoped), Narrative Coherence 0.7 (clearer message delivery than planned).

The system calculates accuracy metrics. Notoriety prediction error: +0.03 (predicted 0.12, actual 0.15, slight underestimation). Respect prediction error: -0.03 (predicted 0.08, actual 0.05, slight overestimation). ROI Index error: -6 points (predicted 62, actual 68). The system flags respect overestimation pattern and prompts the architect to reflect on whether they underestimated relationship friction from assertiveness. Cognitive journal entry links to director's actor profile noting defensive reaction pattern when risk concerns raised, creating actor-specific bias awareness for future interactions.

This complete workflow demonstrates the closed-loop system operation from actor assessment through probabilistic prediction, strategic configuration, empirical outcome capture, accuracy measurement, and continuous model refinement.

The system incorporates a comprehensive training feedback mechanism that quantifies user performance across multiple skill dimensions during interactive communication simulations. This subsystem monitors five key performance indicators providing real-time visibility into skill development progress.

The training progress tracking system automatically increments performance metrics during simulation execution without requiring manual user intervention. Total Interactions Counter increments on simulation completion regardless of outcome quality, establishing training volume baseline for rate calculations. The mathematical representation is $Total_Interactions(t) = Total_Interactions(t-1) + 1$ where t represents current interaction timestamp. This metric is persisted per-architect in the relational database providing cumulative training volume across all historical sessions.

Successful ROI Tracker evaluates whether $Actual_ROI \geq Target_ROI$ generating binary success determination per interaction. The success event is defined as one if $ROI_achieved \geq ROI_target$ and zero otherwise. Cumulative count $Successful_ROI(t) = Successful_ROI(t-1) + Success_Event$ enables success rate calculation. This metric quantifies outcome achievement independent of process quality.

Success Rate Calculator computes percentage as $(Successful_ROI / Total_Interactions) \times 100$ on-demand without persistent storage. The calculation updates in real-time as underlying metrics change, with edge case $Success_Rate = 0$ when $Total_Interactions = 0$ preventing division by zero errors. This derived metric provides performance trending visibility.

Adaptive Pivots Counter triggers when strategy switch occurs mid-interaction detected through conditions $ROI_gap > Pivot_Threshold$ AND $New_Strategy \neq Current_Strategy$. The pivot event = 1 if $|ROI_actual - ROI_target| > threshold\ theta$ AND $Strategy_new \neq$

Strategy_current, otherwise 0. Cumulative count Adaptive_Pivots(t) = Adaptive_Pivots(t-1) + Pivot_Event where threshold theta is inversely proportional to adaptation skill level. This metric quantifies strategic flexibility independent of outcome success, enabling users to track improvement in adaptive thinking even when final outcomes remain suboptimal. This represents a novel contribution distinguishing process skill from result achievement.

Biases Recognized Accumulator tracks cognitive distortion detection events across three bias types: Confirmation bias detected through pattern matching on repeated failed strategies, Anchoring bias detected through unchanged initial estimates despite contradictory evidence, and Halo effect detected through positive actor attribute overgeneralization. Bias events trigger through User_Acknowledgment OR System_Detection mechanisms. Cumulative count Biases_Recognized(t) = Biases_Recognized(t-1) + |Bias_Events_in_interaction| aggregates across all interaction history. This metric separates metacognitive awareness (bias recognition) from task performance (ROI achievement), enabling tracking of self-monitoring skill development as distinct capability. This represents a novel contribution emphasizing metacognitive skills with equal prominence to task execution.

Database event triggers automatically update timestamps on metric changes through PostgreSQL trigger CREATE TRIGGER update_training_metrics_timestamp AFTER UPDATE ON architect_training_metrics FOR EACH ROW EXECUTE FUNCTION update_timestamp(). Additional trigger notifies WebSocket clients through PostgreSQL LISTEN/NOTIFY mechanism: CREATE TRIGGER notify_metric_update AFTER UPDATE ON architect_training_metrics FOR EACH ROW EXECUTE FUNCTION pg_notify('training_updates', row_to_json(NEW)::text).

WebSocket clients subscribe to ws://api/training/{architect_id} endpoint receiving JSON payloads on metric changes enabling UI updates without polling or page refresh. Message format includes architect_id, total_interactions, successful_roi, success_rate, adaptive_pivots, biases_recognized, and timestamp fields. This real-time update architecture provides immediate feedback during training sessions enhancing learning effectiveness.

The Training Progress Footer component displays five metric cards in responsive grid layout. Each card includes integer display for Total Interactions with tooltip "Number of completed communication simulations", integer display for Successful ROI with tooltip "Interactions achieving target outcome", percentage display for Success Rate with tooltip "Percentage of successful interactions", integer display for Adaptive Pivots with tooltip "Strategic adjustments based on feedback", and integer display for Biases Recognized with tooltip "Cognitive biases detected and acknowledged". Visual feedback includes color coding (green for improvement trends, red for decline patterns), animated transitions on value changes, and responsive grid layout adapting to screen size. The interface provides equal visual prominence to metacognitive awareness (biases) alongside task performance (ROI) reinforcing the importance of self-monitoring skills in communication training. This UI design represents a novel contribution in training system visualization emphasizing metacognitive development.

Metrics are cached in-memory using Redis with 60-second time-to-live preventing excessive database reads. Database queries execute only on cache miss events.

Write-through cache strategy ensures immediate consistency on metric updates. Scalability architecture shards metrics by architect_id enabling horizontal scaling. WebSocket connections are pooled and load-balanced across multiple server instances. Batch updates handle high-frequency interaction scenarios without performance degradation.

The system provides comprehensive analytics visualizations enabling architects to track prediction accuracy trends, skill progression patterns, and systematic bias identification over time. These analytics capabilities extend beyond simple metric display to provide actionable insights for targeted skill development.

Figure 13 illustrates the skill progression dashboard displaying charts for each of the three core architect skills (observation, adaptation, self-awareness) tracked across interaction timeline. Each skill chart shows initial baseline value, current effective value after confounder adjustments, and trend line indicating improvement or degradation trajectory. The dashboard highlights periods where specific confounders significantly impacted effective skill levels, enabling architects to identify situational performance patterns. For example, an architect may discover their observation skill degrades by forty percent during high-stress interactions but remains stable during low-stress scenarios, informing pre-interaction preparation strategies focusing on stress management techniques.

The skill progression visualization includes milestone markers indicating significant capability improvements such as achieving consistent eighty percent observation accuracy across ten consecutive interactions, or maintaining adaptation effectiveness above seventy percent despite variable confounder levels. These milestones provide motivational feedback and quantifiable skill development evidence.

Figure 14 illustrates the prediction accuracy analysis panel displaying time-series trends for prediction errors across strategic outcomes (notoriety, respect, wealth). Each outcome includes mean absolute error calculation showing average prediction deviation, trend analysis indicating whether accuracy improves over time through cumulative learning, and confidence interval width tracking showing whether architects develop more precise predictions (narrower intervals) or maintain conservative uncertainty ranges (wider intervals).

The accuracy dashboard identifies systematic bias patterns through statistical analysis of prediction errors. If an architect consistently overestimates change in respect by fifteen to twenty percent across interactions with high-status actors, the system highlights this actor-specific bias pattern with recommendation to recalibrate respect prediction models for status-sensitive scenarios. If wealth predictions consistently underestimate outcomes by twenty-five percent when leveraging high competence strategy, the system identifies competence-wealth correlation underestimation suggesting strategy lever recalibration.

Additionally, the analytics system generates calibration plots comparing predicted outcome probability distributions to actual outcome frequencies. Well-calibrated architects produce predictions where events predicted to occur with seventy percent probability actually occur seventy percent of the time empirically. Poorly calibrated architects show systematic overconfidence (predicted probabilities higher than actual frequencies) or underconfidence (predicted probabilities lower than actual frequencies). The calibration plot enables architects to diagnose and correct these systematic estimation biases.

The skill tracking system also correlates confounder impact patterns with skill degradation rates. Statistical analysis may reveal that for a specific architect, stress level above sixty percent causes observation skill degradation of fifty percent whereas stress below forty percent causes minimal degradation, identifying a nonlinear stress threshold effect. This insight enables architects to implement stress management interventions when approaching critical threshold levels.

These analytics and skill progression capabilities represent high-value novel contributions (novelty assessment eight to nine out of ten) by providing systematic feedback mechanisms enabling quantifiable skill development measurement, bias pattern identification, and targeted improvement recommendations unavailable in prior art training systems.